

TP— 1 et 2

Construction d'un Projet Scientifique Reproductible et Analyse de Stabilité Numérique

Module : Science des Données et Calcul Scientifique Avancé

D1— Session 2026

Table des matières

1	Objectifs Pédagogiques	2
2	Partie 1 : Isolation et Gestion de l'Environnement	2
3	Partie 2 : Configuration et Modularité du Pipeline	3
4	Partie 3 : Définition des Tests Unitaires	4
5	Partie 4 : Gestion du Versionnement Scientifique	4
6	Partie 5 : Intégration dans l'Orchestrator Principal	5
7	Partie 6 : Stabilité Numérique	6
7.1	Exercice 6.1 : Horizon de visibilité numérique et non-associativité	6
7.2	Exercice 6.2 : Le Multiplicateur de Mensonge — Sensibilité au conditionnement .	7
7.3	Exercice 6.3 : Inversion Explicite vs Factorisation Directe	7
8	Questions de Réflexion et Livrables	8
8.1	Questions de Contrôle de Validation (À rédiger sur votre rapport)	8
8.2	Livrables	9

1 Objectifs Pédagogiques

Ce projet de TP se divise en deux volets essentiels de votre formation de chercheur :

1. **Ingénierie de la Reproductibilité** : Structurer un projet Python, isoler son environnement d'exécution, externaliser sa configuration et automatiser ses tests.
2. **Stabilité Numérique et Algèbre Linéaire** : Mettre en pratique les concepts clés du dernier cours (conditionnement, arithmétique IEEE 754, décompositions matricielles) pour prévenir l'effondrement d'un modèle face au bruit numérique.

Objectif d'Apprentissage

À la fin de ce TP, vous devrez soumettre un dépôt Git fonctionnel contenant un code source modulaire, testé, capable de reproduire vos graphiques et métriques en "un clic", tout en garantissant la robustesse mathématique de vos algorithmes de résolution.

2 Partie 1 : Isolation et Gestion de l'Environnement

Votre premier travail consiste à déclarer les dépendances logicielles de votre projet pour éviter le syndrome du *"Ça fonctionne sur ma machine"*.

Travail À Faire (✍)

Complétez le fichier `environment.yml` ci-dessous à la racine de votre dossier de projet. Vous devez configurer l'environnement pour utiliser Python 3.10 et intégrer les paquets scientifiques fondamentaux : `numpy`, `pandas`, `scikit-learn`, `matplotlib` et `pyyaml`.

```
1 # TODO: Complétez les zones manquantes du fichier d'environnement
2 name: air_quality_stability
3 channels:
4   - conda-forge
5   - defaults
6 dependencies:
7   - python=3.10
8   # --- TODO: Ajoutez ici les librairies indispensables ---
9
10
11   - pip
12   - pip:
13     - logging-config
```

Travail À Faire (✍)

Exécutez dans votre terminal les commandes nécessaires pour instancier cet environnement à l'aide de `mamba` (ou `conda`) et l'activer. Écrivez les commandes ci-dessous :

```
1 # 1. Commande de creation de l'environnement a partir du fichier YAML
2 :
3 # TODO:
4
5 # 2. Commande d'activation de l'environnement :
```

3 Partie 2 : Configuration et Modularité du Pipeline

Pour qu'une expérience soit reproductible, les hyperparamètres (chemins de fichiers, graines aléatoires, configuration du modèle) ne doivent jamais être codés "en dur" dans les scripts Python.

Travail À Faire (📌)

Analysez et complétez les sections vides du script de prétraitement `src/preprocessing.py`. Vous devez implémenter l'imputation des valeurs manquantes par la **médiane** et réaliser la coupure des *outliers* par la méthode de l'écart interquartile (IQR).

```

1  """
2  Module de pretraitement des donnees - A COMPL TER
3  """
4  import numpy as np
5  import pandas as pd
6  from sklearn.model_selection import train_test_split
7  from sklearn.preprocessing import StandardScaler
8  from typing import Tuple, Dict, Any
9
10 def clean_data(df: pd.DataFrame, config: Dict[str, Any]) -> pd.
    DataFrame:
11     df_clean = df.copy()
12     handle_missing = config['preprocessing']['handle_missing']
13
14     # 1. Gestion des valeurs manquantes
15     for col in df_clean.columns:
16         if df_clean[col].isnull().any():
17             if handle_missing == 'median':
18                 # TODO: Calculez la mediane et imputez les valeurs
19                     manquantes
20                 pass
21
22     # 2. Gestion des valeurs aberrantes (Outliers via IQR)
23     threshold = config['preprocessing']['outlier_threshold']
24     numeric_cols = df_clean.select_dtypes(include=[np.number]).
25         columns
26
27     for col in numeric_cols:
28         if col != config['data']['target_column']:
29             Q1 = df_clean[col].quantile(0.25)
30             Q3 = df_clean[col].quantile(0.75)
31             IQR = Q3 - Q1
32             # TODO: Définissez les bornes inferieure (lower) et
33                 superieure (upper)
34             lower = # TODO
35             upper = # TODO
36
37             # Application du clipping pour neutraliser les outliers
38             df_clean[col] = df_clean[col].clip(lower, upper)
39
40     return df_clean

```

4 Partie 3 : Définition des Tests Unitaires

La validation de la reproductibilité logicielle passe par l'écriture de tests unitaires automatisés s'assurant du comportement invariant de vos blocs algorithmiques.

Travail À Faire (✍️)

Complétez le script de test `tests/test_preprocessing.py` en utilisant le framework `pytest`. Vous devez écrire un test qui vérifie que deux appels consécutifs à la fonction de partitionnement des données produisent **exactement** le même résultat lorsque la graine de reproductibilité (*seed*) est fixée.

```
1 import numpy as np
2 import pandas as pd
3 import pytest
4 from src.preprocessing import split_and_scale
5
6 def test_split_reproducibility():
7     """Vérifie l'invariance du partitionnement via la graine
8         aléatoire."""
9     # Génération d'un jeu de données de test fictif
10    df = pd.DataFrame({
11        'feature1': range(100),
12        'feature2': range(100),
13        'pm25': range(100)
14    })
15
16    config = {
17        'data': {'target_column': 'pm25', 'test_size': 0.2},
18        'project': {'random_seed': 42},
19        'preprocessing': {'scale_features': True}
20    }
21
22    # TODO: Réalisez deux appels successifs à split_and_scale avec la
23    #       même config
24    # et utilisez np.testing.assert_array_equal pour valider l'
25    #       identité des matrices.
26
27    # X_train1, X_test1, _, _, _ = split_and_scale(df, config)
28    # X_train2, X_test2, _, _, _ = split_and_scale(df, config)
29
30    # TODO: Insérez vos assertions ici
```

5 Partie 4 : Gestion du Versionnement Scientifique

Un dépôt Git de recherche ne doit jamais stocker de gros volumes de données brutes, de fichiers de caches ou de checkpoints volumineux sous peine de saturer le serveur.

Travail À Faire (✍️)

Rédigez le contenu du fichier `.gitignore` à placer à la racine de votre projet afin d'exclure les fichiers de données brutes se trouvant dans le sous-dossier `data/`, tout en conservant l'architecture de vos répertoires à l'aide des fichiers de préservation `.gitkeep`.

```
1 # TODO: Ecrivez les regles d'exclusion de votre .gitignore
   scientifique
2 # Exclure les caches Python :
3
4
5 # Exclure le contenu des repertoires de donnees (conserver uniquement
   les fichiers d'architecture)
6
7
8 # Exclure les checkpoints Jupyter Notebooks :
```

6 Partie 5 : Intégration dans l'Orchestrateur Principal

Le script `src/main.py` est le chef d'orchestre unifié de votre projet de recherche.

Travail À Faire (✍️)

Complétez la fonction principale du fichier `src/main.py` pour y intégrer le chargement de la configuration externalisée YAML et s'assurer du basculement du moteur graphique Matplotlib en mode non-interactif ('Agg') pour éviter les erreurs d'affichage en environnement serveur ou conteneurisé.

```
1 import sys
2 import os
3 import matplotlib
4 # TODO: Basculez matplotlib sur le backend non-interactif 'Agg'
5 # matplotlib.use(...)
6
7 import matplotlib.pyplot as plt
8 from src.utils import load_config, set_seed
9
10 def main():
11     # TODO: Chargez le fichier de configuration configs/config.yaml
12     config = # TODO
13
14     # Fixation universelle des graines pour la reproductibilite
15     set_seed(config['project']['random_seed'])
16
17     print("[RUN] Execution du pipeline scientifique...")
18     # Le reste du code execute les etapes de pretraitement et d'
19     apprentissage...
20
21 if __name__ == "__main__":
22     main()
```

7 Partie 6 : Stabilité Numérique

Objectif d'Apprentissage

Ce TP d'application directe de notre dernier cours de cours magistral vise à vous faire toucher du doigt la fracture épistémologique entre la théorie mathématique en précision infinie ($\mathbb{R}$) et la réalité physique de l'arithmétique des ordinateurs (Standard IEEE 754).

7.1 Exercice 6.1 : Horizon de visibilité numérique et non-associativité

On cherche à observer l'accumulation des erreurs d'arrondis lors de la manipulation de nombres d'ordres de grandeur très différents.

Travail À Faire (✍)

Complétez le script Python ci-dessous pour implémenter la méthode de sommation naïve d'un tableau de flottants et comparez-la à la **sommation de Kahan** vue en cours. Complétez les lignes de calcul manquantes.

```
1 import numpy as np
2
3 def sommation_naive(tableau):
4     somme = 0.0
5     for x in tableau:
6         somme += x
7     return somme
8
9 def sommation_kahan(tableau):
10    somme = 0.0
11    c = 0.0 # Une variable de compensation pour accumuler les petits
           arrondis perdus
12    for x in tableau:
13        # TODO: Implementez l'algorithme de Kahan vu en cours
14        y = x - c
15        t = somme + y
16        c = (t - somme) - y # Compensation de l'erreur d'arrondi
17        somme = t
18    return somme
19
20 # --- Verification experimentale ---
21 epsilon_machine = np.finfo(float).eps
22 print(f"Epsilon Machine de votre processeur : {epsilon_machine}")
23
24 # On cree un tableau contenant un grand nombre suivi de beaucoup de
   petites valeurs
25 valeurs = [1.0] + [epsilon_machine / 2.0] * 10000
26
27 print(f"Somme Naive : {sommation_naive(valeurs)}")
28 print(f"Somme Kahan : {sommation_kahan(valeurs)}")
```

7.2 Exercice 6.2 : Le Multiplicateur de Mensonge — Sensibilité au conditionnement

On étudie la résolution du système linéaire $Ax = b$, où A est une matrice mal conditionnée. On va analyser comment une infime perturbation de l'ordre de grandeur du bruit machine (10^{-14}) sur le vecteur de mesures b modifie radicalement le vecteur de prédiction x .

Travail À Faire (✍)

Complétez le script de perturbation matricielle. Calculez le conditionnement de la matrice A à l'aide de la fonction appropriée de NumPy, résolvez les deux systèmes (exact et perturbé), et calculez l'erreur relative induite sur la solution.

```
1 import numpy as np
2
3 # Generation d'une matrice mal conditionnee (Ex: Matrice de Hilbert
  #      modifiee)
4 A = np.array([[1.0, 1.0/2.0, 1.0/3.0],
5               [1.0/2.0, 1.0/3.0, 1.0/4.0],
6               [1.0/3.0, 1.0/4.0, 1.0/5.0]])
7
8 b = np.array([1.0, 0.5, 0.3333])
9
10 # 1. Calcul du conditionnement
11 # TODO: Utilisez la fonction numpy appropriée pour evaluer kappa(A)
  #      en norme 2
12 kappa_A = # TODO
13
14 print(f"Conditionnement de la matrice de resolution kappa(A) = {
  kappa_A}")
15
16 # 2. Resolution du systeme exact
17 # TODO: Utilisez le solveur lineaire stable de numpy (Bannir
  #      explicitement .inv())
18 x_exact = # TODO
19
20 # 3. Injection d'une perturbation microscopique sur le vecteur b
21 b_perturbe = b.copy()
22 b_perturbe[2] += 1e-14 # Perturbation de l'ordre du bruit de mesure
23
24 # Resolution du systeme avec perturbation
25 x_perturbe = # TODO
26
27 # 4. Calcul de l'erreur relative sur la solution finale
28 erreur_relative = np.linalg.norm(x_exact - x_perturbe) / np.linalg.
  norm(x_exact)
29 print(f"Erreur relative induite sur la prediction x : {
  erreur_relative:.6f}")
```

7.3 Exercice 6.3 : Inversion Explicite vs Factorisation Directe

Dans cet exercice, nous allons quantifier expérimentalement pourquoi le cours recommande de **bannir l'utilisation de l'inversion explicite** d'une matrice.

Travail À Faire (✍)

Complétez le code ci-dessous pour mesurer le temps d'exécution et la norme du résidu géométrique $\|Ax - b\|$ pour deux approches de résolution d'un système linéaire de grande taille ($n = 4000$) : l'approche par calcul explicite de l'inverse ($A^{-1}b$) et l'approche par factorisation directe ('np.linalg.solve').

```
1 import numpy as np
2 import time
3
4 # Generation d'un systeme lineaire dense de grande taille
5 n = 4000
6 rng = np.random.RandomState(42)
7 A = rng.rand(n, n)
8 b = rng.rand(n)
9
10 # --- Approche A : Mauvaise pratique (Inversion explicite) ---
11 t0 = time.time()
12 # TODO: Calculez x en inversant explicitement la matrice A
13 x_inv = # TODO
14 t_inv = time.time() - t0
15 residu_inv = np.linalg.norm(A @ x_inv - b)
16
17 # --- Approche B : Bonne pratique (Factorisation stable) ---
18 t0 = time.time()
19 # TODO: Calculez x en utilisant le solveur direct stable de numpy
20 x_solve = # TODO
21 t_solve = time.time() - t0
22 residu_solve = np.linalg.norm(A @ x_solve - b)
23
24 print(f"Inversion Explicite -> Temps: {t_inv:.4f}s | Norme du residu:
25       {residu_inv}")
26 print(f"Solveur Direct -> Temps: {t_solve:.4f}s | Norme du
27       residu: {residu_solve}")
```

8 Questions de Réflexion et Livrables

8.1 Questions de Contrôle de Validation (À rédiger sur votre rapport)

1. **Analyse Algorithmique** : Dans l'exercice 6.3, quel a été le ratio de gain de temps entre le solveur direct et l'inversion explicite? Justifiez ce résultat à partir des complexités d'opérations en virgule flottante (Flops) vues en cours.
2. **Interprétation Mathématique** : Si une matrice possède un conditionnement de $\kappa(A) = 10^{13}$, et que votre machine travaille avec un epsilon machine de $\epsilon_{mach} \approx 10^{-16}$, combien de chiffres significatifs de votre vecteur solution x sont mathématiquement exacts? Que représentent les autres chiffres?
3. **Éthique et Rigueur** : Pourquoi la modification directe, manuelle et non-documentée d'un fichier de données brutes est-elle considérée comme une faute de méthode scientifique majeure?

8.2 Livrables

Vous devez nous envoyer sur le groupe de D1 l'adresse URL de votre dépôt Git public contenant :

- ☐ L'architecture complète des dossiers respectant le schéma cible.
- ☐ Le fichier `environment.yml` complété et valide.
- ☐ Les fichiers sources de la librairie `src/` exempts de tout bug.
- ☐ Le fichier de configuration `configs/config.yaml`.
- ☐ Un fichier `README.md` expliquant la commande unique permettant d'exécuter l'intégralité du pipeline et des tests.